# Atividade Prática 2: Paradigmas de Programação

**Nome dos integrantes:** Gabriel Machado, Hyan Salas, Hatus Gabriel, Pabline Pereira, Gilberto Borges

## PARTE 1 - Antes de comparar linguagens

**1. O que é um paradigma de programação?**
De forma bem simples, um paradigma é como uma "forma de pensar" ou um guia de regras na hora de resolver um problema no computador. É como se fosse a planta baixa de uma casa: antes de começar a programar de fato, o paradigma define qual será a estrutura básica da nossa lógica e como vamos organizar as informações.
**2. Linguagem e paradigma são a mesma coisa?**
Não. O paradigma é a forma de pensar e estruturar a solução (a ideia), enquanto a linguagem é a ferramenta que usamos para colocar essa ideia em prática. Você pode pensar no paradigma como o "estilo musical" (ex: rock) e a linguagem como o "instrumento" (ex: guitarra).
**3. Uma mesma linguagem pode utilizar mais de um paradigma?**
Sim, muitas linguagens modernas são multiparadigma. O Python, por exemplo, permite que você escreva um código usando o paradigma imperativo (sequência de passos), orientado a objetos (criando classes e objetos) ou funcional (focado em funções matemáticas).
**4. Por que a arquitetura de von Neumann teve tanta influência sobre as primeiras linguagens de programação?**
Porque as primeiras linguagens precisavam "conversar" diretamente com a forma física como os computadores foram construídos. Na arquitetura de von Neumann, os dados e os programas ficam no mesmo lugar (memória). Assim, as linguagens precisaram criar o conceito de "variáveis" para representar esses espaços de memória, e a execução do programa passou a ser uma sequência passo a passo de "instruções" (pegar o dado na memória, processar na UCP e devolver), criando o que chamamos de execução sequencial.

## PARTE 2 - Conhecendo as três linguagens

| Característica | C | Java | Python |
|---|---|---|---|
| **Ano aproximado de criação** | Anos 1970 (1972). | Anos 1990 (1995). | Anos 1990 (1991). |
| **Principal utilização atualmente** | Sistemas operacionais e programas diretos no hardware. | Sistemas de grandes empresas e aplicativos Android. | Inteligência artificial, análise de dados e automações. |
| **Paradigmas suportados** | Focado totalmente no Imperativo/Procedural. | Orientado a Objetos fortemente, com toques funcionais. | Multiparadigma (Orientado a objetos, imperativo e funcional). |
| **Tipagem** | Estática (precisa avisar o tipo) e fraca. | Estática e forte (muito rigorosa com os tipos). | Dinâmica (descobre o tipo na hora) e forte. |
| **Compilada ou interpretada?** | Compilada direto para código de máquina. | Compilada para bytecode e rodada na máquina virtual (JVM). | Interpretada linha por linha. |
| **Possui tratamento de exceções?** | Não. Usa códigos de erro numéricos antigos. | Sim, usa blocos try/catch muito rígidos. | Sim, usa blocos try/except bem flexíveis. |
| **Gerenciamento automático de memória?** | Não. O programador tem que limpar a memória na mão. | Sim. Tem o "Garbage Collector" que limpa a memória. | Sim, também gerencia tudo sozinho por baixo dos panos. |
| **Exemplo de** | O sistema de freios de um carro ou um | O sistema de um | Uma inteligência artificial de |

| Característica | C | Java | Python |
|---|---|---|---|
| **sistema** | Arduino. | banco. | recomendação de filmes. |

# PARTE 3 - Colocando o framework de Sebesta em prática

## Matriz de avaliação de Sebesta

| Critério | C | Java | Python |
|---|---|---|---|
| **Legibilidade** | 2 | 3 | 5 |
| **Capacidade de escrita** | 2 | 3 | 5 |
| **Confiabilidade** | 2 | 5 | 3 |
| **Custo** | 2 | 4 | 5 |
| **TOTAL** | **8** | **15** | **18** |

## Justificativas das notas:

**5. Legibilidade**

- **C (nota 2):** A sintaxe é mais antiga e o uso de ponteiros e regras de memória deixa o código muito sujo e difícil de ler rápido.
- **Java (nota 3):** É muito bem estruturada, mas é excessivamente longa (verbosa). Você precisa ler muitas linhas de configuração para entender uma ideia simples.
- **Python (nota 5):** Parece inglês simples. A leitura é fluida, usa indentação (espaços) e

não tem excesso de chaves ou símbolos confusos.

**6. Capacidade de escrita**

- **C (nota 2):** Exige muito trabalho manual. O programador gasta mais tempo gerenciando o computador do que focando no problema real.
- **Java (nota 3):** Tem muitas ferramentas boas e orientação a objetos pura, mas obriga a escrever muitos "moldes" (classes) antes de fazer o código funcionar.
- **Python (nota 5):** É super expressivo. O que leva 10 linhas em Java, você escreve em 2 ou 3 no Python, focando totalmente no resultado.

**7. Confiabilidade**

- **C (nota 2):** Não protege o sistema. Se você errar o acesso à memória (aliasing indesejado), o programa inteiro quebra e trava de forma silenciosa.
- **Java (nota 5):** É um "tanque de guerra". Tem verificação de tipos fortíssima e um excelente tratamento de exceções que impede o sistema de desmoronar por erros bobos.
- **Python (nota 3):** Como a tipagem é dinâmica (os tipos podem mudar no meio do caminho), é mais fácil passar algum erro despercebido que só vai estourar quando o usuário estiver usando.

**8. Custo**

- **C (nota 2):** Programadores C são difíceis de achar, o treinamento é demorado e arrumar bugs de memória custa muito tempo (e dinheiro).
- **Java (nota 4):** Apesar de demorar um pouco mais para escrever, o custo de manutenção de longo prazo é excelente para sistemas gigantes.
- **Python (nota 5):** É muito fácil de aprender, o time desenvolve o programa super rápido, e há muitos profissionais no mercado hoje.

# PARTE 4 - Análise dos pequenos códigos

**9. O que os três programas fazem?**

Eles fazem um loop que vai do número 1 até o 5, somando cada um desses números em uma variável chamada "total" e, no final, imprimem o resultado (que será 15) na tela.

**10. Qual código foi mais fácil de compreender? Explique por quê.**

O Código C (Python). Ele parece uma frase em linguagem humana: "para o número no intervalo de 1 a 6, some ao total e depois imprima". Não tem ponto e vírgula, chaves soltas ou declaração repetitiva de tipos.

**11. Qual deles parece exigir mais conhecimento da linguagem para ser entendido?**

O Código A (em C) e o Código B (em Java), pois exigem que o leitor saiba o que significa int, o que faz o operador ++, para que servem as chaves {} delimitando os blocos e o porquê do %d

na hora de imprimir no C.

**12. É possível dizer que menos linhas de código significa automaticamente melhor legibilidade? Justifique.**

Nem sempre. Às vezes, um programador tenta espremer muita lógica em uma linha só (o que chamamos de falta de ortogonalidade), virando um código quase impossível de decifrar. Porém, no caso do Python do exemplo, ele tem menos linhas porque tem maior abstração e expressividade, não por estar apertado.

**13. Algum desses exemplos permite perceber características do paradigma imperativo?**

Sim, todos eles mostram a cara do paradigma imperativo, que espelha a máquina. As três características observadas são:

1. Controle de estado: A variável total tem seu valor alterado (mutação) a cada passo.
2. Execução sequencial: O código roda determinística e linearmente de cima para baixo.
3. Lógica do "como fazer": Nós dizemos explicitamente como o computador deve repetir (fazer um laço passo a passo) para chegar no resultado.

# PARTE 5 - Escolha da linguagem para o sistema de clínicas

**14. Entre C, Java e Python, qual linguagem vocês escolheriam?**

Java.

**15. Apresentem três argumentos a favor da linguagem escolhida:**

1. Como o projeto vai durar pelo menos 10 anos, a tipagem forte e a estrutura do Java facilitam muito a manutenção a longo prazo.
2. O tratamento de erros (exceções) do Java garante alta confiabilidade, algo essencial lidando com prontuários e vidas de pacientes.
3. É ótimo para sistemas com muitos acessos simultâneos (50 usuários) e integração segura com redes corporativas.

**16. Apresentem duas desvantagens da própria escolha:**

1. A capacidade de escrita é mais lenta: a equipe vai demorar mais tempo para lançar a primeira versão se comparado ao Python.
2. Vai exigir servidores mais potentes (maior custo de infraestrutura), pois a JVM e os sistemas Java costumam consumir mais memória.

**17. Qual linguagem vocês considerariam menos adequada para esse projeto? Justifiquem.**

Linguagem C. Para um sistema de gestão em clínicas, usar C seria terrível. A linguagem não tem abstração de alto nível para interfaces modernas e o programador perderia meses gerenciando ponteiros e memória à toa, tornando o custo do projeto inviável e correndo

grandes riscos sistêmicos.

**18. Dos quatro critérios de Sebesta, qual deveria receber maior peso nesse sistema?**

A Confiabilidade. Um sistema médico mexe com dados de faturamento e laudos de pacientes. Uma falha aqui não é apenas um "bug", é um problema legal e de saúde. O software precisa tratar exceções para não travar e perder dados no meio de uma consulta.

# PARTE 6 - Escolha da linguagem para o sistema embarcado

**19. Vocês manteriam a linguagem escolhida anteriormente? Sim ou não? Por quê?**

Não. O Java usa uma máquina virtual (JVM) que consome muita memória e tem processos automáticos de limpeza (Garbage Collector) que podem dar pequenas pausas na execução. Num sistema embarcado industrial de tempo real e memória limitada, isso causaria acidentes e travamentos.

**20. Qual linguagem escolheriam agora? Justifiquem.**

Linguagem C. Ela conversa diretamente com a máquina física, consome um mínimo de memória e fornece velocidade máxima, pois não tem "camadas" escondendo as coisas. O programador tem controle total sobre o hardware.

**21. O que essa mudança demonstra sobre a pergunta "Qual é a melhor linguagem de programação?"**

Demonstra que a "melhor" linguagem não existe de forma absoluta. Uma linguagem incrível para gestão médica (Java) falhou miseravelmente na fábrica, enquanto a linguagem "ruim" para clínicas (C) brilhou no hardware. Tudo depende dos critérios do projeto.

# PARTE 7 - Conclusão

A escolha de uma linguagem de programação vai muito além da preferência pessoal; ela exige analisar o problema e os critérios de Sebesta no mundo real. Para sistemas longos, a **confiabilidade** é vital: linguagens com **tipagem** rigorosa evitam desastres em produção e diminuem o custo de **manutenção** ao longo da década. Por outro lado, para iniciar rapidamente, focar na **capacidade de escrita** e **legibilidade** pode ser mais sensato, priorizando a velocidade da equipe. Além disso, o **paradigma** da linguagem altera o pensamento: modelar o mundo real exige **abstração** em Orientação a Objetos, enquanto manipular o hardware direto prefere uma abordagem imperativa. Portanto, a melhor linguagem é a que equilibra riscos e benefícios técnicos para cada contexto específico.